

Лабораторная работа №5: Продвинутые возможности

Часть 1: Продвинутая работа с исключениями

1.1 Иерархия исключений и создание собственных исключений

Базовый класс для исключений приложения

```
class LibraryError(Exception):
    """Базовое исключение для библиотечной системы"""
    pass

class BookNotFoundError(LibraryError):
    """Книга не найдена"""
    def __init__(self, book_id):
        self.book_id = book_id
        super().__init__(f"Книга с ID {book_id} не найдена")

class BookNotAvailableError(LibraryError):
    """Книга недоступна"""
    def __init__(self, title):
        super().__init__(f"Книга '{title}' сейчас выдана")

class UserNotFoundError(LibraryError):
    pass
```

Пример использования

```
def borrow_book(book_id, user_id):
    # ... проверки ...
    if not book_exists(book_id):
        raise BookNotFoundError(book_id)
    if not is_book_available(book_id):
        raise BookNotAvailableError(get_book_title(book_id))
    # ... выдача ...

try:
    borrow_book(123, 456)
except BookNotFoundError as e:
    print(f"Ошибка: {e}")
except BookNotAvailableError as e:
    print(f"Книгу нельзя выдать: {e}")
```

1.2 Блок else и finally, цепочки исключений

```

def divide_numbers(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Деление на ноль!")
        raise # повторно возбуждаем исключение
    else:
        print("Деление выполнено успешно")
        return result
    finally:
        print("Блок finally выполняется всегда")

# Цепочка исключений (raise ... from ...)
def process_file(filename):
    try:
        with open(filename) as f:
            return int(f.read())
    except FileNotFoundError as e:
        raise ValueError("Не удалось обработать файл") from e

try:
    process_file("missing.txt")
except ValueError as e:
    print(f"Ошибка: {e}")
    print(f"Причина: {e.__cause__}") # оригинальное исключение

```

Задание 1.1: Разработайте иерархию исключений для банковской системы (например, `BankError`, `InsufficientFundsError`, `AccountNotFoundError`, `InvalidAmountError`). Напишите функцию перевода денег между счетами, которая использует эти исключения.

Часть 2: Итераторы

Итератор – объект, который поддерживает метод `__next__()` и выбрасывает `StopIteration`, когда элементы заканчиваются.

```

# Класс-итератор для чисел Фибоначчи
class Fibonacci:
    def __init__(self, max_count):
        self.max_count = max_count

```

```

self.count = 0
self.a, self.b = 0, 1

def __iter__(self):
    # Итератор должен возвращать себя
    return self

def __next__(self):
    if self.count >= self.max_count:
        raise StopIteration
    result = self.a
    self.a, self.b = self.b, self.a + self.b
    self.count += 1
    return result

fib = Fibonacci(10)
for num in fib:
    print(num, end=" ") # 0 1 1 2 3 5 8 13 21 34

```

Бесконечный итератор (не реализует `StopIteration`) – использовать осторожно.

```

class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

```

Задание 2.1: Создайте класс `Cycle`, который принимает список и бесконечно циклически возвращает его элементы (как `itertools.cycle`). Реализуйте методы `__iter__` и `__next__`. Продемонстрируйте работу, взяв первые 10 элементов.

Часть 3: Генераторы

Генераторы – простой способ создания итераторов с помощью ключевого слова `yield`.

3.1 Функции-генераторы

```
def fibonacci_gen(max_count):
    a, b = 0, 1
    for _ in range(max_count):
        yield a
        a, b = b, a + b

for num in fibonacci_gen(10):
    print(num, end=" ") # 0 1 1 2 3 5 8 13 21 34

# Генератор с yield from (делегирование)
def chain(*iterables):
    for it in iterables:
        yield from it

combined = chain([1, 2, 3], (4, 5), "abc")
print(list(combined)) # [1, 2, 3, 4, 5, 'a', 'b', 'c']
```

3.2 Генераторные выражения

Как списковые включения, но в круглых скобках и создают генератор.

```
# Список квадратов чисел
squares_list = [x**2 for x in range(10)] # сразу вычисляет все элементы
squares_gen = (x**2 for x in range(10)) # генератор, элементы вычисляются по
# требованию

print(sum(x**2 for x in range(10))) # 285 (без создания промежуточного списка)
```

3.3 Бесконечные генераторы и взаимодействие с генератором

```
def read_lines():
    """Имитация чтения большого файла построчно"""
    for i in range(1000000):
        yield f"Строка {i}"

# Обработка без загрузки всего в память
lines = read_lines()
for _ in range(5):
```

```

    print(next(lines))

# Передача данных в генератор (сопрограммы) – через .send()
def echo():
    while True:
        received = yield
        print(f"Получено: {received}")

gen = echo()
next(gen)          # "запускаем" генератор до первого yield
gen.send("Привет") # Получено: Привет
gen.send(123)      # Получено: 123
gen.close()

```

Задание 3.1: Напишите генератор `read_large_file(filename)`, который читает файл построчно и возвращает строки, удаляя пробельные символы по краям и пропуская пустые строки. Протестируйте на небольшом текстовом файле.

Часть 4: Декораторы

Декораторы – функции, которые принимают другую функцию и расширяют её поведение без изменения кода.

4.1 Простые декораторы

```

def timer(func):
    """Измеряет время выполнения функции"""
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} выполнялась {end - start:.4f} сек")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
    print("Функция завершена")

```

```
slow_function()
```

4.2 Декораторы с параметрами

```
def repeat(n):
    """Декоратор, повторяющий вызов функции n раз"""
    def decorator(func):
        def wrapper(*args, **kwargs):
            for i in range(n):
                print(f"Вызов {i+1}:")
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def say_hello():
    print("Привет!")

say_hello()
```

4.3 Декораторы для методов классов

```
def method_info(func):
    def wrapper(self, *args, **kwargs):
        print(f"Вызов метода {func.__name__} объекта {self}")
        return func(self, *args, **kwargs)
    return wrapper

class MyClass:
    @method_info
    def do_something(self):
        print("Что-то делаем")

obj = MyClass()
obj.do_something()
```

4.4 functools.wraps – сохранение метаданных функции

```
from functools import wraps
```

```
def my_decorator(func):
    @wraps(func) # без этого __name__, __doc__ будут от wrapper'a
    def wrapper(*args, **kwargs):
        """Документация wrapper'a"""
        print("Что-то до")
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def example():
    """Документация примера"""
    pass

print(example.__name__) # example (а не wrapper)
print(example.__doc__) # Документация примера (а не wrapper'a)
```

Задание 4.1: Создайте декоратор `log_calls`, который записывает в файл `log.txt` имя функции, время вызова и переданные аргументы. Используйте `functools.wraps`. Примените к нескольким функциям.

Часть 5: Контекстные менеджеры

Контекстный менеджер определяет поведение при входе в блок `with` и выходе из него.

5.1 Класс-контекстный менеджер

```
class ManagedFile:
    def __init__(self, filename, mode='r'):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode, encoding='utf-8')
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        # Если вернуть True, исключение не пробрасывается дальше
        # Например, можно обработать исключение и вернуть True
```

```
        return False

with ManagedFile('test.txt', 'w') as f:
    f.write("Привет, мир!")
```

5.2 Использование contextlib

```
from contextlib import contextmanager

@contextmanager
def managed_file(filename, mode):
    try:
        f = open(filename, mode, encoding='utf-8')
        yield f
    finally:
        f.close()

with managed_file('test.txt', 'a') as f:
    f.write("Ещё строка\n")
```

5.3 Вложенные контекстные менеджеры

```
from contextlib import contextmanager

@contextmanager
def tag(name):
    print(f"<{name}>", end="")
    yield
    print(f"</{name}>", end="")

with tag('b'):
    with tag('i'):
        print(" Текст ", end="")

# Выведет: <b><i> Текст </i></b>
```

Задание 5.1: Создайте контекстный менеджер `Timer`, который замеряет время выполнения блока кода и выводит его (например, при выходе из блока). Реализуйте двумя способами: через класс и через `@contextmanager`.

Часть 6: Применение декораторов и контекстных менеджеров в реальных задачах

6.1 Декоратор для повторных попыток (retry)

```
import time
from functools import wraps

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Попытка {attempt} не удалась: {e}")
                    if attempt == max_attempts:
                        raise
                    time.sleep(delay)
            return wrapper
        return decorator

@retry(max_attempts=3, delay=2)
def unstable_network_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("Сбой сети")
    return "Успех"

print(unstable_network_call())
```

6.2 Контекстный менеджер для временного изменения директории

```
import os
from contextlib import contextmanager

@contextmanager
def change_dir(new_dir):
    old_dir = os.getcwd()
    try:
        os.chdir(new_dir)
```

```
        yield
    finally:
        os.chdir(old_dir)

with change_dir('/tmp'):
    print(os.listdir('.')) # список файлов в /tmp
print(os.getcwd()) # вернулись обратно
```

Задание 6.1 (комплексное): Напишите декоратор `@cache` (или используйте `functools.lru_cache`), который запоминает результаты вызова функции для заданных аргументов. Реализуйте свой простой вариант (например, с использованием словаря). Затем примените к рекурсивной функции вычисления чисел Фибоначчи и сравните скорость с и без кэширования.